MeTTa Review Tool — Test Catalogue

72 tests · 18 unit · 51 integration · 3 E2E

6 July 2026

Unit tests (18) — pure logic, no database

`tests/unit/logic.test.ts`

clampPosition — position clamping / resume math

Test	Verifies
clamps into [0, total-1]	normal in-range positions map to themselves; out-of-range clamps to ends
handles empty totals, NaN, Infinity, fractional	0 total → 0; NaN → 0; +Infinity → last; fractional floored

verdict derivation

Test	Verifies
correct means both yes	a Correct row reconstructs to premiseOk+hypothesisOk true
failed flags invert to ok booleans	Failed flags map correctly to the Yes/No the reviewer chose
isCorrect is true only when both yes	routing: both Yes = Correct, anything else = Failed

normalizeNote

Test	Verifies
trims and nullifies empties	whitespace/empty/undefined note → null; real note trimmed

computeStats

Test	Verifies
derives every field	reviewed/toFix/gold/premise & hypothesis Yes-No all computed correctly
is safe when nothing reviewed	zero reviews → no negatives / NaN / divide-by-zero

input validation (zod)

Test	Verifies
accepts valid input	well-formed verdict input passes
defaults note to empty string	omitted note defaults to ""

Test	Verifies
rejects empty id, non-boolean flags, and oversized note	bad annotationId / wrong types / >4000-char note rejected
fixInput requires id + non-empty metta	fix input needs failedId and non-empty MeTTa both sides
failedIdInput validates the id	empty failedId rejected

`tests/unit/export.test.ts`

toCsv — CSV serialisation

Test	Verifies
returns empty string for no rows	empty input → empty CSV
writes a header row + data rows	header + one line per row
handles null/undefined, booleans, Dates as ISO	null → empty cell; booleans stringified; Dates → ISO
escapes quotes, preserves commas/newlines in quoted cells	RFC-style quoting (`""` escaping)
serializes nested objects as JSON	object cells become JSON strings

Integration tests (51) — real server actions against Postgres

`tests/integration/review.test.ts`

submitVerdict

Test	Verifies
both Yes → one Correct + one Final(DIRECT), no Failed	happy path writes the right rows
premise No only → Failed(premiseFailed=true, hypothesisFailed=false)	which-side-failed flags
hypothesis No only → Failed(false, true)	which-side-failed flags
both No → Failed(true, true)	both sides flagged
is idempotent (submit twice keeps one row)	re-submitting the same verdict doesn't duplicate
transition Yes → No removes Correct/Final, creates Failed	changing a verdict cleans up the old rows
transition No → Yes removes Failed (and cascades), creates Correct/Final	reverse transition, incl. cascading away a fix
unknown annotationId → error, no writes	invalid target rejected atomically
invalid input (empty id) → error, no writes	validation blocks bad input

getReviewData

Test	Verifies
total=0 → null annotation	empty dataset handled
resumes at the first unreviewed pair	resume-where-left-off
prefills an existing verdict when navigating back	edit-in-place shows the prior verdict
clamps an out-of-range position to the last pair	boundary navigation

`tests/integration/failed.test.ts`

submitFix / revertFix

Test	Verifies
writes Fixed(original+fixed) + Final(FIXED), flags isFixed	fixing produces the right rows + lineage
re-fixing replaces the prior fix (no duplicates)	idempotent fix
empty fixed MeTTa → error, no writes	validation on the fix
unknown failedId → error	invalid target rejected
revert removes Fixed + Final(FIXED), clears isFixed	undo a fix cleanly
getFailedList returns the row with its fix data	list surfaces the fix
re-verdicting a fixed pair to Correct cleans up Fixed + Final(FIXED)	cross-action cascade

`tests/integration/dashboard.test.ts`

getStats

Test	Verifies
counts a mixed review set correctly	correct/failed/fixed/gold + by-side numbers on a real mix
is safe with zero reviews	empty state → zeros, no errors

`tests/integration/invariants.test.ts`

data-integrity invariants

Test	Verifies
a pair is never in both Correct and Failed at once	mutual exclusivity holds across transitions
count(Final) == count(Correct) + count(Fixed), no orphan Final(FIXED)	Final is exactly Correct + Fixed
every fixed Failed row has isFixed = true	fix flag consistency

database constraints

Test	Verifies
one verdict per (annotation, reviewer) — duplicate rejected	composite-unique enforced
two different reviewers can each review the same annotation	per-reviewer verdicts allowed
deleting an Annotation cascades to all verdict tables	FK cascade
deleting a Failed row cascades to its Fixed row	FK cascade
Final.origin rejects values outside the enum	enum constraint

`tests/integration/multi-reviewer.test.ts`

multi-reviewer + attribution

Test	Verifies
stamps reviewerId + reviewerName on Correct and Final	attribution recorded
stamps reviewerName on Failed	attribution recorded
stamps reviewerName on Fixed	attribution recorded
two reviewers review the SAME annotation independently	no conflict between reviewers
resume position is per-reviewer	each reviewer resumes at their own spot

Test	Verifies
one reviewer changing a verdict does not affect the other	isolation

`tests/integration/edge-cases.test.ts`

edge cases, concurrency, scoping, security

Test	Verifies
stores unicode + embedded quotes in MeTTa without corruption	encoding safety
rejects an oversized note (validation) and writes nothing	input limits
handles many parallel submits (same reviewer, different pairs)	parallelism / pooling
a reviewer only sees their own data (scoping / no IDOR)	read authorisation
clamps navigation at both boundaries	boundary safety
empty dataset returns a null annotation, not an error	empty state
stores a note on a correct verdict (Correct + Final)	notes on correct pairs
prefills the stored note when navigating back to a correct pair	note is viewable/editable
a reviewer cannot fix or revert another reviewer's failed row (IDOR)	write authorisation (security)
stores a note on a fix (Fixed + Final)	fix notes

`tests/integration/stress.test.ts`

stress / load

Test	Verifies
handles a burst of 40 parallel submits without loss or corruption	throughput + integrity under load (drove the pool-size fix)

E2E tests (3) — Playwright / Chromium (real browser, test-mode auth)

`e2e/review.spec.ts`

Test	Verifies
review a pair, see progress update, then reach export	full UI flow: render → click Yes/Yes → Save → progress
export page lists tables with download links	export page renders with JSON/CSV links
dashboard renders	dashboard page loads

Run: `npm test` (unit), `TEST_DATABASE_URL=… npm run test:integration`, `… npm run test:e2e`.